

## RakshaNet

- 01 Executive Verdict
- 02 Feature Audit
- 03 SIH Judge Evaluation
- 04 Top 10 Weaknesses
- 05 Recommendations
- 06 USP Positioning
- 07 Demo Flow
- 08 MVP vs Future
- 09 Action Roadmap
- 10 Final Verdict

---

P0 findings: 1  
Weaknesses logged: 10  
Recommendations: 7  
Engines are real, deterministic – zero ML claims

---

TECHNICAL & PRODUCT AUDIT – READ-ONLY REVIEW, NO CODE CHANGED

## Is RakshaNet actually a winning system, or does it just look like one?

A full-stack audit of the backend engines, the web Command Center, and the Expo mobile app — evaluated as an SIH judge, a disaster-management domain reviewer, and a systems architect would each read it.

**Scope** backend · web · mobile · database · docs · tests

**Method** file-by-file evidence, no assumptions

## 01 Executive Verdict

---

### THE HEADLINE

RakshaNet is **unusually complete for a hackathon build** — a real PostGIS-backed Postgres schema with 16 tables, four deterministic scoring engines, roughly 45 fully-implemented API routes with no stub handlers found anywhere, a React Command Center with consistently-implemented loading/error/empty states across every page, and a genuinely working Expo mobile app with real offline SQLite sync, idempotent retry/backoff, and real camera capture. This is not a UI mockup wired to fake data. On technical depth alone, it would out-engineer most SIH submissions.

### THE PROBLEM

The single workflow step the whole PRD is built around — **Report** → **Verify** → **Risk/HVI Update** — is currently broken in the live app. Verifying a hazard report never recomputes the affected habitation's HVI, and the Command Center has **no button anywhere** that triggers a recalculation either. Confirmed directly:

`HabitationDrawer.jsx:75` tells the officer to *"Run a risk recalculation to generate one,"* but no control in the entire web app calls `POST /risk/recalculate`. After the seed script runs once, every HVI/zone value is frozen until someone calls the API outside the UI. This is exactly the moment a judge is most likely to be watching for, and right now nothing visibly happens.

### THE FIX IS SMALL; THE STORY PROBLEM IS NOT

Technically, this is a one-function, low-complexity fix (see [P0 roadmap](#)). The bigger risk is narrative: the project currently has two parallel "mobile" experiences (the real Expo app, and a browser-based `/mobile-preview` inside the web app) with feature drift

between them, and none of the "AI" language in the UI is backed by a trained model — which is fine and honestly disclosed in the code, but needs to be framed deliberately rather than discovered by a skeptical judge.

Net read: this is a strong, technically credible submission with **one demo-breaking wiring bug and a positioning problem** — not a features problem. Fixing the risk-recompute wiring both repairs the core story and creates the demo's best wow moment in one move.

## 02 Current Feature Audit

Every row is grounded in code inspected across `backend/` , `web/` , `mobile/` , and `database/seed.js` . Nothing here is assumed missing without a matching grep.

| MODULE / FEATURE                       | STATUS | EVIDENCE                                                                                                                                             |
|----------------------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Citizen hazard/vulnerability reporting | ✓ Well | <code>mobile/src/screens/shared/ReportFormScreen.js</code> ; real multipart photo upload                                                             |
| SOS emergency alert                    | ✓ Well | <code>sos.controller.js</code> , real-time <code>sos:new</code> socket event                                                                         |
| Offline queue & sync (mobile)          | ✓ Well | <code>mobile/src/db/database.js</code> , exponential backoff, <code>local_uuid</code> idempotency                                                    |
| Volunteer verification workflow        | ✓ Well | Dual surface: <code>VerificationQueuePage.jsx</code> (officer) + <code>TasksScreen.js</code> (mobile) both hit <code>POST /reports/:id/verify</code> |
| Field Survey (HVI checklist)           | ✓ Well | <code>mobile/src/screens/volunteer/FieldSurveyScreen.js</code> , feeds <code>housingStructuralFactor</code>                                          |
| Risk/HVI scoring engine (logic)        | ✓ Well | <code>riskHvi.engine.js</code> — 5 weighted factors, full history table, transparent breakdown                                                       |
|                                        |        |                                                                                                                                                      |

| MODULE / FEATURE                         | STATUS                                                                                              | EVIDENCE                                                                                                                                           |
|------------------------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Risk/HVI live recompute trigger</b>   |  <b>Missing</b>    | No call site invokes recompute on verify; <code>recalculateRisk</code> defined in <code>endpoints.js:19</code> but never called from any component |
| GIS Live Risk Map                        |  <b>Well</b>       | <code>RiskMapPage.jsx</code> — real Leaflet, zone-colored markers, hazard-type filter, street/satellite toggle                                     |
| Satellite imagery cross-check            |  <b>Well</b>       | <code>SatelliteThumb.jsx</code> — real Esri World Imagery, used in 4 places, explicitly labeled non-AI                                             |
| Safe-site management / screening         |  <b>Well</b>       | <code>SafeSiteDrawer.jsx</code> , <code>safeSites.controller.js</code>                                                                             |
| Carrying-capacity / resource sufficiency |  <b>Well</b>       | <code>capacity.engine.js</code> — per-resource ratios, Surplus/Adequate/Critical, "idle site" edge case unit-tested                                |
| Resource redistribution suggestion       |  <b>Needs work</b> | <code>suggestRedistribution()</code> in <code>capacity.engine.js</code> — greedy match, suggestion-only                                            |
| Site matching / ranking engine           |  <b>Well</b>     | <code>matching.engine.js</code> — 6-factor weighted score, "why this site" rationale, unit-tested                                                  |
| Split-population allocation              |  <b>Well</b>     | <code>splitAllocationAcrossCandidates()</code> , seeded Periyar scenario                                                                           |
| Stress test before approval              |  <b>Well</b>     | <code>runStressTest()</code> , blocks approval unless explicitly overridden                                                                        |
| Human approval workflow                  |  <b>Well</b>     | Transactional approve: plan status + site occupancy + <code>ResponseTask</code> creation in one DB transaction                                     |
| Route status & alternate routing         |  <b>Well</b>     | Seeded blocked route with linked alternate ( <code>seed.js:219-231</code> )                                                                        |
| Response task tracking                   |  <b>Well</b>     | Assigned → En Route → On Site → Resolved state machine, <code>responseTasks.controller.js</code>                                                   |
| Dashboard KPI summary                    |  <b>Well</b>     | <code>dashboard.controller.js</code> — all 8 KPIs computed live, no hardcoding                                                                     |
|                                          |                                                                                                     |                                                                                                                                                    |

| MODULE / FEATURE                               | STATUS       | EVIDENCE                                                                                                                        |
|------------------------------------------------|--------------|---------------------------------------------------------------------------------------------------------------------------------|
| Reports & export (CSV/PDF/DOCX)                | ✔ Well       | Real <code>pdfkit</code> / <code>docx</code> generation, not stubbed                                                            |
| Audit log                                      | ⚠ Needs work | <code>audit.service.js</code> called from only 3 controllers                                                                    |
| Role-based access control                      | ✔ Well       | Backend-enforced on every route; only <code>/audit</code> is route-blocked client-side, rest hide controls                      |
| Auth (OTP + email/password + quick-start)      | ✔ Well       | <code>LoginPage.jsx</code> random demo-account button; real Twilio fallback to dev OTP                                          |
| SMS/IVR low-connectivity intake                | ⚠ Needs work | <code>smsIvr.service.js</code> , honestly labeled <code>mode: 'demo-mock'</code>                                                |
| Fake-report / duplicate moderation             | ⚠ Needs work | <code>moderation.service.js</code> — heuristic, computed, self-labeled <code>heuristic-v1</code>                                |
| Digital Twin / environmental sensor simulation | ✖ Missing    | Zero matches for "digital twin," "soil moisture," "tilt," "vibration," "sensor" across the entire repo (3 independent searches) |
| "What-if" scenario simulation                  | ✖ Missing    | Stress test is capacity-only, single-plan, not a general simulator                                                              |
| Real-time updates — Command Center             | ✔ Well       | 15 socket events emitted, all subscribed to across web pages                                                                    |
| Real-time updates — mobile app                 | ✖ Missing    | <code>socket.io-client</code> in <code>package.json</code> , zero actual usage in <code>mobile/src</code>                       |
| Automated test coverage                        | ⚠ Needs work | 5 pure-logic unit suites + 1 DB-free validation suite; zero integration/E2E tests                                               |

| MODULE / FEATURE                              | STATUS         | EVIDENCE                                                                                     |
|-----------------------------------------------|----------------|----------------------------------------------------------------------------------------------|
| Loading / error / empty states                | ✓ Well         | Verified consistent across 8 web pages + mobile <code>StateViews.js</code>                   |
| Terminology consistency                       | ✓ Well         | Centralized <code>ROLE_LABELS</code> / <code>ZONE_LABELS</code> , no drift found anywhere    |
| No hardcoded demo values                      | ✓ Well         | <code>DemoDataBadge</code> pattern; dashboard summary explicitly computed live               |
| Seeded demo scenarios (required 5)            | ✓ Well         | All 5 confirmed present in <code>database/seed.js</code> (see §3.6)                          |
| Push notifications (FCM)                      | — Not required | Explicitly deferred in code comments                                                         |
| Two parallel mobile surfaces w/ feature drift | ⚠ Needs work   | Web preview has Duty Status + Dispatch Comms; neither exists in the PRD or the real Expo app |
| Standalone frontend deployability             | ✓ Well         | Verified: <code>web/</code> installs and builds with zero monorepo dependency                |

## 03 SIH Judge Evaluation

### 1. Problem understanding

Clear and specific. The data model itself — not just the pitch deck — encodes the full government decision chain: habitation → risk → safe site → capacity → match → plan → approval → response. That's the actual process an SDMA/DDMA officer follows, not a generic "report a problem" civic-tech shape.

### 2. Innovation

What's genuinely innovative: capacity-aware, stress-tested, *split-allocation* relocation matching. Most disaster apps stop at "report a hazard" or "show a risk map." Almost none check whether the destination shelter can actually hold the incoming population before recommending it, split people across multiple shelters when one isn't enough, and re-validate the whole plan with a stress test before a human signs off. That's the real differentiator.

What feels generic in isolation: the reporting/SOS/map layer. A judge who only sees the first 90 seconds — mobile app, SOS button, a map — could reasonably mistake this for "yet another disaster reporting app," because the differentiated part only shows up once you reach the Relocation Planner. **Demo order determines whether the USP lands.**

### 3. USP

Strongest genuine USP: capacity-validated, split-allocation relocation planning with mandatory human approval. Not currently obvious from the UI within the first minute — see §6 for the repositioning.

### 4. Technical depth

Real engineering throughout: PostGIS spatial queries ( `ST_DWithin` , `ST_Distance` ), transactional approve-cascades, offline-first mobile sync with idempotency and exponential backoff, real satellite tiles, real file exports (PDF/DOCX, not stub downloads). Nothing found to be purely decorative UI. The two places that are simulated are both honestly labeled in their own source comments: SMS/IVR ( `mode: 'demo-mock'` without real Twilio/Exotel credentials) and the fake-report moderation heuristic (self-tagged `heuristic-v1` ) — the latter's real flaw isn't dishonesty, it's that its output is never used downstream.

### 5. AI credibility

Nothing in this codebase is a trained ML model. Every "AI-assisted" score is a hand-coded weighted sum or rule table: `HVI_WEIGHTS` , `MATCH_WEIGHTS` , `HOUSING_RISK` , `ROAD_ACCESS_RISK` , `ROUTE_RELIABILITY_SCORE` , and fixed resource-status thresholds. The engine's own comment says it plainly: *"Deliberately kept as explainable arithmetic (no black-box ML) so it stays swappable for a real model later."* The one line of UI copy that uses the word — *"AI only recommends"* ( `RelocationPlannerPage.jsx:64` ) — is immediately paired with visible factor/weight breakdowns everywhere a score appears.

This is a credibility **strength** if pitched as "explainable decision support," and a credibility **risk** only if the pitch deck oversells it as machine learning. Recommendation: say the word

"algorithmic" or "rule-based decision support" once, early, in the pitch — then immediately show the transparent factor breakdown as proof. Turning the honesty into the selling point ("we show our work — no black box") pre-empts the "is this really AI?" question rather than getting caught by it.

## 6. Real-world feasibility

Plausible as a district-level SDMA/DDMA pilot MVP. Data dependency that would matter at real scale: every habitation and safe site is currently officer-entered one at a time through a form — there's no bulk import or GIS shapefile ingestion, which would be a genuine blocker if a state tried to onboard its full village inventory. That's correctly out of scope for a hackathon MVP, but worth naming proactively rather than waiting for a judge to ask. Nothing else in the build is over-scoped for what it claims.

## 7. UI/UX

No confusing screens, no overcrowded pages, consistent terminology, consistent loading/error/empty states verified on 8 pages. The only duplicate functionality — report verification existing on both the officer dashboard and the volunteer mobile app — is a deliberate, correct design (officers verify from the Command Center, volunteers verify from the field), not an accidental doubling. The one dilution risk is the web preview's invented Duty Status and Dispatch Comms features, which map to nothing in the PRD or the real app.

## 8. End-to-end flow

Mostly connected, with one critical break. Downstream of a (currently manual, currently button-less) risk recompute, the chain is real: risk zone escalates → habitation flagged Red/Orange → "Generate Relocation Plan" → matching engine ranks sites → split allocation if needed → stress test → officer approves → occupancy updates and a response task is created in the same transaction → responder advances status → dashboard KPIs live-update via socket. That entire back half is genuinely wired and demoable. **The one broken link is the step judges will be watching for first: verify → visible risk change.**

## 9. Demo impact — wow-moment gaps

Existing wow-moment candidates: the transactional approve → capacity update → response-task creation happening live with a socket-pushed dashboard change; the split-allocation UI on the seeded Periyar scenario; the stress-test block on capacity overload; the satellite cross-check

inside a verify decision. What's currently missing a wow moment is exactly the step that should have the biggest one: watching a habitation's zone visibly escalate as reports come in and get verified. That moment doesn't exist right now because the wiring doesn't exist — fixing the P0 bug creates the missing wow moment as a side effect, not a separate task.

## 04 Top 10 Weaknesses

---

### Risk/HVI never recomputes live

W1

#### CURRENT EVIDENCE

`verifications.controller.js` never imports `riskHvi.engine.js` ;  
`HabitationDrawer.jsx:75` tells the officer to "run a risk recalculation" while no button in the app calls `recalculateRisk()` from `endpoints.js:19` — confirmed by direct grep, not agent report alone.

#### WHY IT HURTS THE PROJECT

This is the load-bearing step of the entire pitched workflow. A judge watching a verify action expecting a zone to change will see nothing happen.

#### RECOMMENDED FIX

Call `computeHviForHabitation` (or the cheaper `recomputeAllHvi()` , safe at seed-data scale per the engine's own comment) after a verification decision and after a habitation attribute edit. Add a visible "Recalculate Risk" control on the Risk Map / Habitation Drawer wired to the endpoint that already exists.

Complexity: Low

Impact: High

Priority: P0

### SMS/IVR inbound webhook has no authentication

W2

#### CURRENT EVIDENCE

`POST /api/sms-ivr/inbound` mounted with no JWT/secret check, by design per its own route comment — anyone who finds the URL can POST fabricated hazard reports.

#### WHY IT HURTS THE PROJECT

A technical judge or security reviewer probing the API surface will find this immediately; it undercuts the "human-in-the-loop, trustworthy data" story.

#### RECOMMENDED FIX

Require a shared-secret header (or Twilio/Exotel request-signature validation) before accepting the webhook.

Complexity: Low

Impact: Medium

Priority: P1

## Fake-report moderation is computed but never enforced

W3

#### CURRENT EVIDENCE

`moderation.service.js` writes `isSuspectedFake` / `isDuplicate` onto every report; no query or UI anywhere reads those fields back.

#### WHY IT HURTS THE PROJECT

Dead computation is harmless by itself, but if the pitch ever claims "AI fraud detection," this is the gap a judge would find on inspection — the flag exists but does nothing.

#### RECOMMENDED FIX

Surface it as a small chip in the Verification Queue ("possible duplicate") — cheap, and turns existing dead code into a visible feature.

Complexity: Low

Impact: Medium

Priority: P1

## No automated integration or end-to-end tests

W4

#### CURRENT EVIDENCE

5 pure-logic unit suites plus one DB-free validation/auth slice; the validation test file's own header comment states full CRUD is "exercised manually via docs/DEMO\_SCRIPT.md against a live Postgres instance."

#### WHY IT HURTS THE PROJECT

Unlikely to sink a demo, but a "have you tested this end to end" question currently has a weak answer.

#### RECOMMENDED FIX

One integration test covering Report → Verify → Recompute → Plan → Approve against a real test database.

Complexity: Medium

Impact: Low-Medium

Priority: P2

## Audit log has partial coverage

W5

### CURRENT EVIDENCE

`audit.service.js` is called from only 3 controllers (plan approve/modify/reject, user role/status change, dashboard registration) — login, verification decisions, SOS status changes, and safe-site edits are privileged but unlogged.

### WHY IT HURTS THE PROJECT

Undermines the "accountable human-in-the-loop" pitch, which is core to the disaster-domain credibility story.

### RECOMMENDED FIX

Add `recordAudit` calls to the missing four action types.

Complexity: Low

Impact: Medium

Priority: P1

## Two mobile surfaces, unexplained on screen

W6

### CURRENT EVIDENCE

The real Expo app ( `/mobile` ) and the browser-based `MobilePreviewPage.jsx` both exist; nothing on either surface tells a visiting judge which one is the product and which is a demo convenience.

### WHY IT HURTS THE PROJECT

A judge cross-referencing the pitch against two different-looking "mobile apps" may read it as unfinished/redundant work rather than a deliberate reliability decision (built specifically to route around Expo Go device-compatibility friction).

### RECOMMENDED FIX

One banner line on `/mobile-preview` : "Browser demo of the Citizen/Volunteer experience — the production app is a native Expo build with offline sync and camera capture."

Complexity: Low

Impact: Medium

Priority: P1

## Web preview has features the real product doesn't

W7

### CURRENT EVIDENCE

Duty Status toggle and "Dispatch Comms" quick-message exist only in `MobilePreviewPage.jsx` — not in `docs/PRD.md`, not in the real Expo app.

#### WHY IT HURTS THE PROJECT

A sharp judge comparing the pitch deck's PRD against the live demo could catch the mismatch and start wondering what else was embellished.

#### RECOMMENDED FIX

Either relabel them explicitly as "demo conveniences, not shipped features" or drop them — five-minute fix either way.

Complexity: Low

Impact: Low-Medium

Priority: P2

## Habitation attribute edits don't trigger recompute either

W8

#### CURRENT EVIDENCE

`habitations.controller.js` `updateHabitation` can change `housingType` / `historicalIncidentCount` — direct HVI inputs — without recomputing anything.

#### WHY IT HURTS THE PROJECT

Same root cause family as W1; compounds the staleness problem from a second angle.

#### RECOMMENDED FIX

Same fix as W1, applied in the same controller-update pass.

Complexity: Low

Impact: Medium

Priority: P1

## No bulk/GIS import for habitations or safe sites

W9

#### CURRENT EVIDENCE

Every habitation/safe-site row is created one-at-a-time via `POST /habitations` / `POST /safe-sites` forms; no shapefile/CSV ingestion path exists.

#### WHY IT HURTS THE PROJECT

Not a demo problem — a real-adoption-credibility problem if a judge asks "how would a state actually onboard its 10,000 villages?"

#### RECOMMENDED FIX

Correctly out of scope for the hackathon; name it proactively as a known next step rather than waiting to be asked.

Complexity: Medium

Impact: Low (demo) / High (real adoption)

Priority: P2

Mobile app has no real-time push

W10

CURRENT EVIDENCE

socket.io-client is declared in mobile/package.json but grep across mobile/src finds zero actual connection code.

WHY IT HURTS THE PROJECT

Field volunteers only see updates on manual refresh/app-foreground — weakens the "live coordinated response" claim specifically on the field side.

RECOMMENDED FIX

Wire at minimum the SOS and alert events; full parity with the web isn't necessary for a demo.

Complexity: Medium

Impact: Low-Medium

Priority: P2

05

Winning-Worthy Recommendations

Seven recommendations, all of which either repair or directly strengthen the existing capacity-aware relocation USP. Nothing here is a net-new module — every item wires up code or data that already exists.

| RECOMMENDATION                                               | CURRENT GAP                 | WHY IT IMPROVES WINNING POTENTIAL                                             | PRIORITY | COMPLEXITY |
|--------------------------------------------------------------|-----------------------------|-------------------------------------------------------------------------------|----------|------------|
| Auto-recompute HVI on verify + add a Recalculate Risk button | W1/W8 above                 | Repairs the core PRD workflow and creates the single best wow moment for free | P0       | Low        |
| Add a data-freshness indicator ("HVI                         | No page currently shows how | Makes the staleness question visible and manageable instead of invisible      | P1       | Low        |

| RECOMMENDATION                                                              | CURRENT GAP                                                                | WHY IT IMPROVES WINNING POTENTIAL                                                                                                                                                                                                                   | PRIORITY | COMPLEXITY |
|-----------------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------|
| last calculated...")                                                        | stale a score is                                                           | — most disaster apps never disclose this, so it reads as rigor, not risk                                                                                                                                                                            |          |            |
| Surface the moderation flag in the Verification Queue                       | W3 above — data computed, never shown                                      | Converts already-existing backend work into a visible feature at near-zero cost                                                                                                                                                                     | P1       | Low        |
| Authenticate the SMS/IVR webhook                                            | W2 above                                                                   | Closes a real security gap before a technical judge finds it                                                                                                                                                                                        | P1       | Low        |
| Expand audit logging to verify/SOS/safe-site actions                        | W5 above                                                                   | Strengthens the "accountable human-in-the-loop" pitch, which is central to disaster-domain credibility                                                                                                                                              | P1       | Low        |
| Label the two mobile surfaces explicitly on screen                          | W6/W7 above                                                                | Removes judge confusion between the demo preview and the production app for a one-line cost                                                                                                                                                         | P1       | Low        |
| <b>Read-only "conditions changed" re-stress-test flag on approved plans</b> | PRD question 8 ("what happens if conditions change?") currently unanswered | Reuses the existing <code>runStressTest</code> function against current live data — no new engine, just a scheduled/on-demand re-check with a warning chip on the plan card. The closest honest answer to "what-if" achievable without overbuilding | P1       | Medium     |

## Explicitly rejected, and why

Cascading multi-hazard visualization, family/group-aware relocation, and resource-shortage forecasting were all considered and rejected for now. The current single-hazard filter and zone map already answer "which habitation is at highest risk" clearly — a multi-hazard overlay would look impressive without answering any PRD question better. Family-aware relocation needs a schema change the data model doesn't currently support and solves no demo pain point. Forecasting is already partially answered by the existing `daysOfCoverage` field in the capacity engine — a full model would be over-engineering for an MVP. These are exactly the kind of "sounds impressive" additions the brief warns against.

## 06 Current USP vs. Recommended USP

---

### Current positioning

As named in the PRD: *"AI-driven multi-hazard risk zonation and relocation platform."*

**Weakness** — "AI-driven" invites exactly the scrutiny that reveals there's no trained model behind it (better to get ahead of that than be caught by it), and "multi-hazard risk zonation" foregrounds the part of the product that looks most like every other disaster-reporting app, while burying the part that's actually differentiated three screens deeper.

### Recommended winning USP

Lead with the thing nobody else at this scale does: **validating that a destination can actually hold the people before recommending it** — checking real shelter capacity and resources, splitting the population across sites when one shelter isn't enough, stress-testing the result, and requiring a human to sign off, with every score shown in plain, weighted factors rather than a black box.

#### ONE-SENTENCE VERSION

"RakshaNet doesn't just flag who's at risk — it proves the destination can actually hold them, before any human has to decide."

#### 20-SECOND JUDGE EXPLANATION

"Most disaster apps stop at reporting or a risk map. RakshaNet goes further: once a habitation is flagged high-risk, it checks real shelter capacity and resources against the incoming population, splits people across sites if one shelter isn't enough, stress-tests the result, and shows the officer exactly why each site was chosen — in plain, weighted factors, not a black box. A human always makes the final call; RakshaNet just makes sure they're deciding with the full picture."

## 07 Ideal 3–5 Minute Demo Flow

---

**Scenario: Landslide risk escalation, Periyar Riverside Basti, Idukki.** Built entirely from data already in `database/seed.js` — the Periyar relocation plan is deliberately left at Pending Approval specifically to demo the split-allocation flow live. This script assumes the [PO fix](#) has shipped; without it, step 3 has nothing to show.

### 0:00 Open on the Command Center Overview

Point at the live KPI tiles.

*"This is RakshaNet's live view for Idukki district. Every number here is computed from the database in real time — nothing is hardcoded."*

---

### 0:30 Switch to the Citizen mobile view

Log in as the seeded citizen, submit a new Landslide report near Periyar, severity 4.

*"A citizen near Periyar just reported worsening ground movement."*

---

### 1:15 Switch to the Volunteer view

Open Assigned Tasks, tap the satellite thumbnail to cross-check the exact GPS point, tap Verify.

*"A volunteer on the ground confirms it against a real satellite view of the coordinates — this isn't just trusting the citizen's word."*

---

#### WOW MOMENT

Back on the Command Center, the risk recompute fires (auto or via the new button). Periyar's zone visibly intensifies on the Live Risk Map, and the Habitation Drawer's factor breakdown

updates live. "The moment that report is verified, the risk score recalculates — not a black box, here's exactly which factors moved and why."

2:15

### Open the Relocation Planner

Periyar's plan is already ranked at the top. Open Plan Detail: it's a split allocation across two safe sites, with the weighted "why this site" breakdown for each, and the stress-test result.

*"2,400 people, no single shelter can take them all — RakshaNet splits the allocation and stress-tests both destinations before anyone signs off."*

2:45

### Officer clicks Approve

Show the live cascade: site occupancy updates immediately, a response task appears.

*"The system never moves anyone itself — a human officer approved this. Once approved, everything downstream updates automatically and is fully audit-logged."*

3:15

### Switch to Routes & Response

Show the response task board, advance a task; briefly touch the seeded blocked-route/alternate-route scenario on Rajakkad.

*"And if the direct route is blocked, RakshaNet already has the alternate on file."*

3:45

### Close on the Munnar habitation's completed cycle

Reported → Verified → Risk → Planned → Approved → Resolved, fully done.

*"This is what the full loop looks like end to end — RakshaNet doesn't just plan the move, it tracks it through."*

*"RakshaNet doesn't just flag who's at risk — it proves the destination can actually hold them, before any human has to decide."*

## 08 MVP vs. Should Have vs. Future Scope

---

### MVP — MUST WORK NOW

- Report → Verify → **risk recompute** → zone update
- Live Risk Map with real markers + satellite cross-check
- Safe-site capacity + resource status
- Relocation matching, split allocation, stress test
- Approve/Modify/Reject with occupancy + response-task cascade
- Response task tracking
- Live dashboard KPIs
- All 5 seeded demo scenarios (already present)

### SHOULD HAVE — IF TIME PERMITS

- Data-freshness indicator
- Moderation flag surfaced in Verification Queue
- Expanded audit logging
- On-screen framing for the two mobile surfaces
- Read-only "conditions changed" re-stress-test flag

### FUTURE SCOPE — DO NOT RUSH

- Real digital twin / sensor ingestion
- Bulk/GIS habitation import

- Multi-tenant / multi-state
- A real ML model to augment the weighted scoring
- Push notifications (FCM)
- Full scenario simulator beyond the lightweight re-check
- Family/group-aware relocation
- Resource-shortage forecasting model
- Real-time socket push to the mobile app

## 09 P0 / P1 / P2 Implementation Roadmap

---

### P0 — must fix before presentation

#### Auto-trigger HVI recompute on verification and habitation edits; add a Recalculate Risk control

The engine's own spatial query ( `ST_DWithin` in `hazardExposureFactor` ) doesn't require `report.habitationId` to be set — the simplest correct fix is calling `recomputeAllHvi()` after a verify decision, which the engine's own code comments confirm is cheap at seed-data scale. Then wire `recalculateRisk()` (already defined, unused, in `endpoints.js:19` ) to a real button.

|              |                                                                                                                                                                                                                                                    |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FILES        | <code>backend/src/controllers/verifications.controller.js</code> ,<br><code>backend/src/controllers/habitations.controller.js</code> ,<br><code>web/src/pages/RiskMapPage.jsx</code> or<br><code>components/habitation/HabitationDrawer.jsx</code> |
| DEPENDENCIES | None blocking                                                                                                                                                                                                                                      |
| COMPLEXITY   | Low                                                                                                                                                                                                                                                |
| TEST         | Verify the seeded pending Rajakkad report; confirm <code>currentHvi</code> / <code>currentZone</code> changes without any manual API call. Click the new button; confirm the socket-driven UI updates live.                                        |

### P1 — high-value improvements

## Data-freshness indicator

|            |                                                                                              |
|------------|----------------------------------------------------------------------------------------------|
| FILES      | <code>HabitationDrawer.jsx</code> , <code>RiskMapPage.jsx</code> popups                      |
| COMPLEXITY | Low                                                                                          |
| TEST       | Confirm the "last calculated" label reflects <code>lastCalculatedAt</code> after a recompute |

## Surface moderation flag in Verification Queue

|            |                                                                                         |
|------------|-----------------------------------------------------------------------------------------|
| FILES      | <code>web/src/pages/VerificationQueuePage.jsx</code>                                    |
| COMPLEXITY | Low                                                                                     |
| TEST       | Submit two near-identical reports; confirm the second shows a "possible duplicate" chip |

## Authenticate the SMS/IVR webhook

|            |                                                                                                                 |
|------------|-----------------------------------------------------------------------------------------------------------------|
| FILES      | <code>backend/src/routes/smsIvr.routes.js</code> ,<br><code>backend/src/controllers/smsIvr.controller.js</code> |
| COMPLEXITY | Low                                                                                                             |
| TEST       | Confirm an unsigned request is rejected; a correctly-signed one still creates a report                          |

## Expand audit logging

|            |                                                                                                                                                               |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FILES      | <code>verifications.controller.js</code> , <code>sos.controller.js</code> ,<br><code>safeSites.controller.js</code> , <code>auth.controller.js</code> (login) |
| COMPLEXITY | Low                                                                                                                                                           |
| TEST       | Perform each action; confirm a matching row appears in <code>GET /audit-logs</code>                                                                           |

## On-screen framing for the two mobile surfaces

|            |                                                  |
|------------|--------------------------------------------------|
| FILES      | <code>web/src/pages/MobilePreviewPage.jsx</code> |
| COMPLEXITY | Low                                              |
| TEST       | Visual check only                                |

### Read-only "conditions changed" re-stress-test flag

|              |                                                                                                                                                                                                                                                          |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FILES        | <code>backend/src/engines/relocationPlanner.engine.js</code> (reuse <code>runStressTest</code> ),<br>a new lightweight recheck endpoint in <code>relocationPlans.controller.js</code> ,<br><code>PlanCard.jsx</code> / <code>PlanDetailDrawer.jsx</code> |
| DEPENDENCIES | P0 fix should land first so the underlying HVI/capacity data is trustworthy                                                                                                                                                                              |
| COMPLEXITY   | Medium                                                                                                                                                                                                                                                   |
| TEST         | Approve a plan, artificially reduce the destination site's capacity, confirm the flag appears on next view                                                                                                                                               |

## P2 — only if time remains

### One integration test for the core chain

|            |                                                                             |
|------------|-----------------------------------------------------------------------------|
| FILES      | new file under <code>tests/integration/</code> against a real test database |
| COMPLEXITY | Medium                                                                      |

### Wire mobile app to Socket.io for SOS/alert push

|            |                                                                           |
|------------|---------------------------------------------------------------------------|
| FILES      | <code>mobile/src/context/NetworkContext.js</code> or a new socket context |
| COMPLEXITY | Medium                                                                    |

## 10 Final Verdict

---

What would it take for RakshaNet to feel like a winning project? Almost nothing structural — the foundation is already there, which is rarer than it sounds at this level. What it takes is **one**

**afternoon of wiring, not one more week of building:** reconnect verification to risk recomputation, put a button where the UI already promises one exists, and stop letting the demo start with the least differentiated screen in the product.

Every other gap found here is small, named, and fixable in under a day each — an unauthenticated webhook, a dead moderation flag, thin audit coverage, two mobile surfaces that need one sentence of context. None of them require new engineering. The team already built the hard part: a real, deterministic, explainable, capacity-aware relocation engine with split allocation and mandatory human approval, running against real geospatial data. That is genuinely rare at this level of competition, and it's currently the least visible thing about the demo.

Fix the one wiring bug, tell one connected incident story instead of touring disconnected screens, and let the honesty about "no black-box AI, here's the math" become the pitch's strength instead of something to dodge. That is what turns a technically strong build into a project a judging panel remembers.